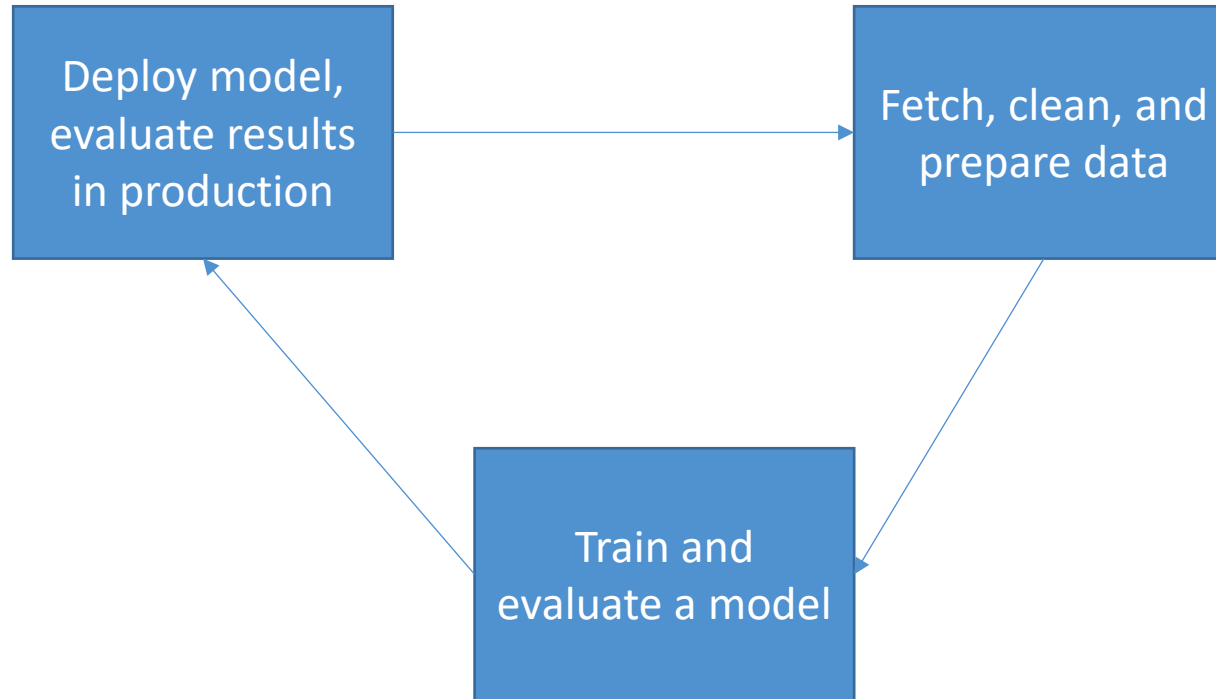
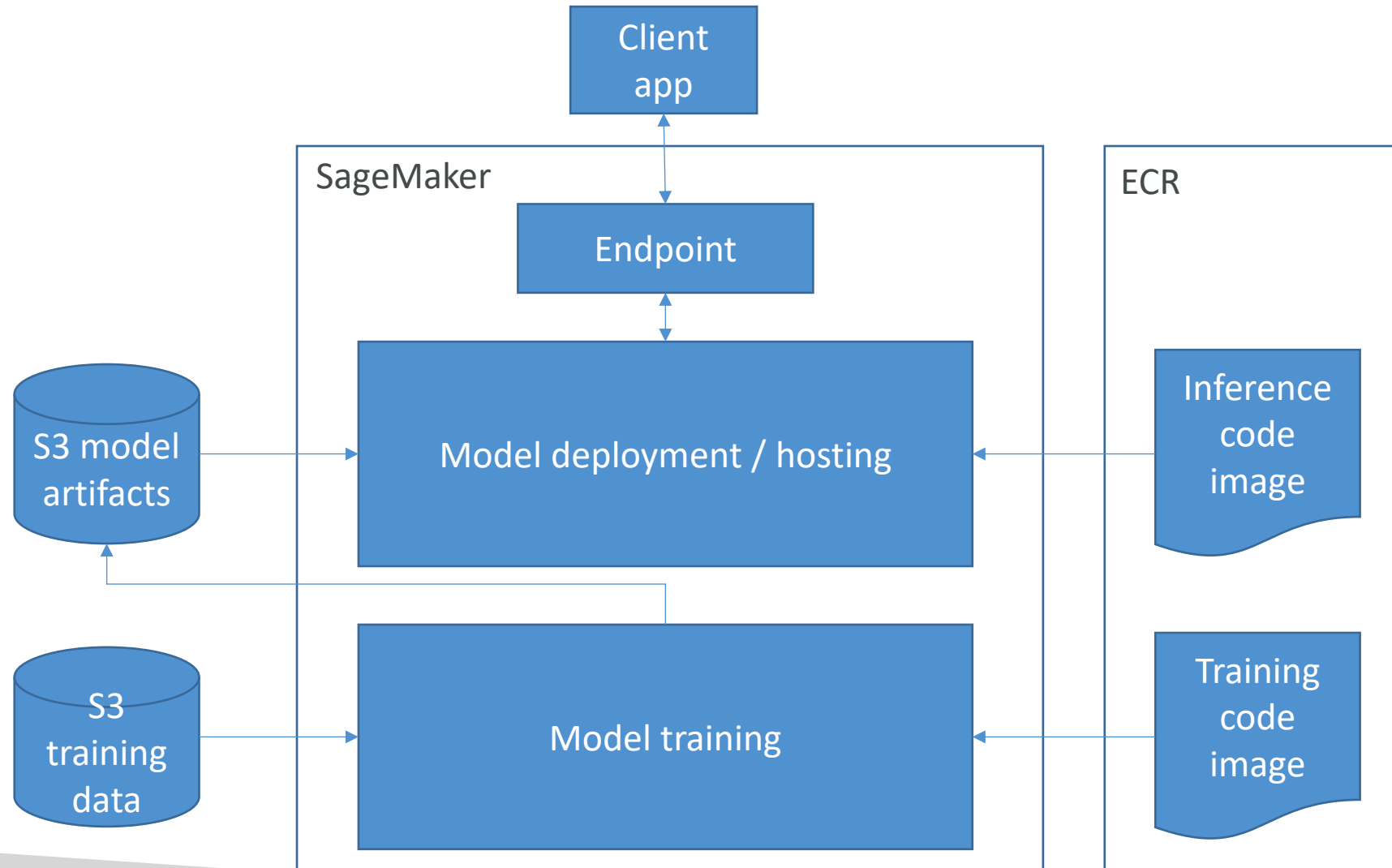


Amazon SageMaker

SageMaker is built to handle the entire machine learning workflow.

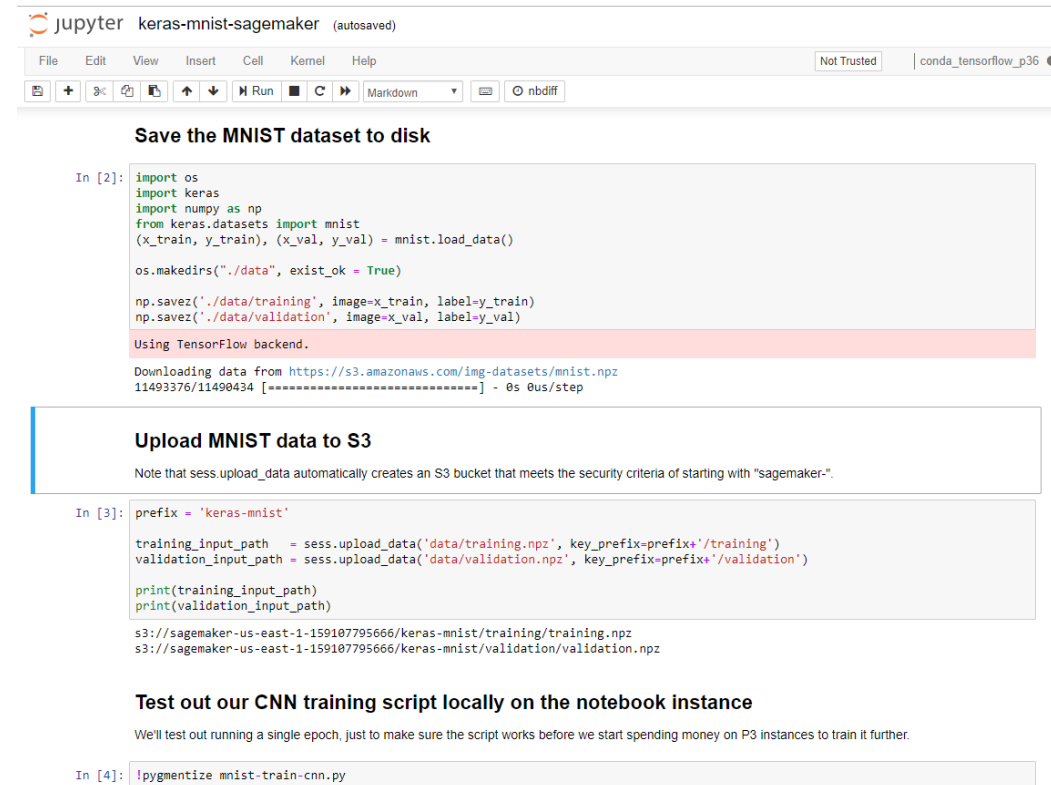


SageMaker Training & Deployment



SageMaker Notebooks can direct the process

- Notebook Instances on EC2 are spun up from the console
 - S3 data access
 - Scikit_learn, Spark, Tensorflow
 - Wide variety of built-in models
 - Ability to spin up training instances
 - Ability to deploy trained models for making predictions at scale



The screenshot shows a Jupyter Notebook interface with the title 'keras-mnist-sagemaker (autosaved)'. The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Help) and a toolbar with icons for file operations and execution. The notebook content is divided into sections:

- Save the MNIST dataset to disk**: This section contains a code cell (In [2]:) that imports necessary libraries (os, keras, numpy, mnist) and saves the MNIST dataset to local files. The code includes comments and a status bar indicating the use of the TensorFlow backend and the download progress of data from S3.
- Upload MNIST data to S3**: This section contains a code cell (In [3]:) that uploads the saved training and validation data to an S3 bucket. It includes a note about the automatic creation of an S3 bucket and the use of the 'sagemaker-' prefix.
- Test out our CNN training script locally on the notebook instance**: This section contains a code cell (In [4]:) that runs a command to execute a training script.

So can the SageMaker console

The screenshot displays the Amazon SageMaker console interface. The top navigation bar includes the AWS logo, 'Services', 'Resource Groups', and user information (Frank Kane, N. Virginia, Support). The left sidebar shows the 'Amazon SageMaker' menu with options like Dashboard, Search, Ground Truth, Notebook, Training, and Inference. The 'Training' section is expanded, showing 'Algorithms', 'Training jobs' (highlighted), and 'Hyperparameter tuning jobs'. The main content area, titled 'Training jobs', features a search bar and a table of training jobs. The table has columns for Name, Creation time, Duration, and Status. All listed jobs are in a 'Completed' status.

Name	Creation time	Duration	Status
sagemaker-tensorflow-190926-1513-010-6f1bf0d5	Sep 26, 2019 15:30 UTC	4 minutes	Completed
sagemaker-tensorflow-190926-1513-009-53c7332e	Sep 26, 2019 15:30 UTC	3 minutes	Completed
sagemaker-tensorflow-190926-1513-008-a53612ca	Sep 26, 2019 15:26 UTC	4 minutes	Completed
sagemaker-tensorflow-190926-1513-007-b06a4d5b	Sep 26, 2019 15:26 UTC	4 minutes	Completed
sagemaker-tensorflow-190926-1513-006-167e1e3c	Sep 26, 2019 15:22 UTC	3 minutes	Completed
sagemaker-tensorflow-190926-1513-005-fd89504c	Sep 26, 2019 15:21 UTC	5 minutes	Completed
sagemaker-tensorflow-190926-1513-004-4a31fc9c	Sep 26, 2019 15:17 UTC	4 minutes	Completed
sagemaker-tensorflow-190926-1513-003-187d6ee7	Sep 26, 2019 15:17 UTC	5 minutes	Completed
sagemaker-tensorflow-190926-1513-002-0fd0ad85	Sep 26, 2019 15:14 UTC	4 minutes	Completed
sagemaker-tensorflow-190926-1513-001-7f7cf798	Sep 26, 2019 15:14 UTC	3 minutes	Completed

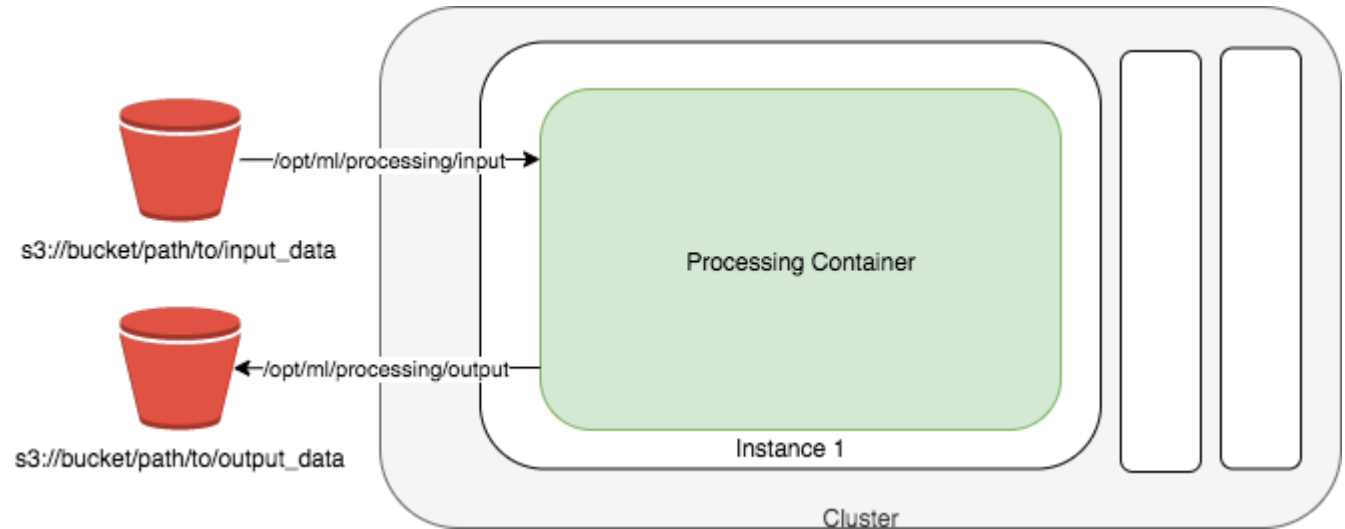
Data prep on SageMaker

- Data usually comes from S3
 - Ideal format varies with algorithm – often it is RecordIO / Protobuf
- Can also ingest from Athena, EMR, Redshift, and Amazon Keyspaces DB
- Apache Spark integrates with SageMaker
 - More on this later...
- Scikit_learn, numpy, pandas all at your disposal within a notebook



SageMaker Processing

- Processing jobs
 - Copy data from S3
 - Spin up a processing container
 - SageMaker built-in or user provided
 - Output processed data to S3



Training on SageMaker

- Create a training job
 - URL of S3 bucket with training data
 - ML compute resources
 - URL of S3 bucket for output
 - ECR path to training code
- Training options
 - Built-in training algorithms
 - Spark MLlib
 - Custom Python Tensorflow / MXNet code
 - PyTorch, Scikit-Learn, RLEstimator
 - XGBoost, Hugging Face, Chainer
 - Your own Docker image
 - Algorithm purchased from AWS marketplace



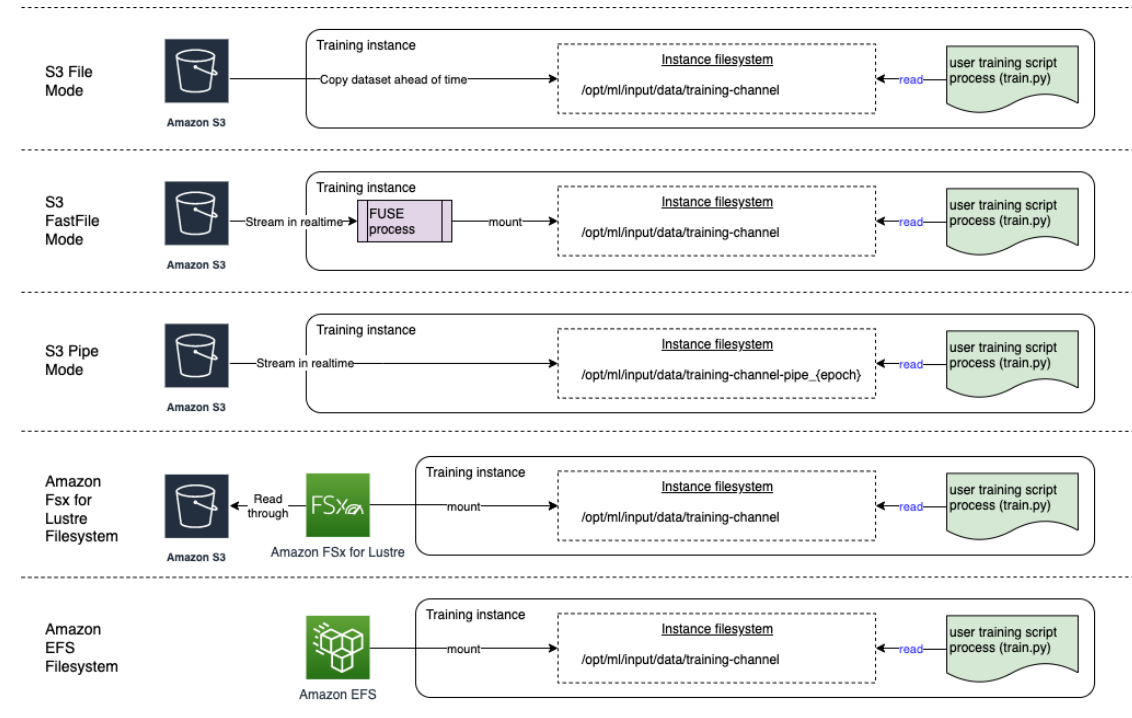
Deploying Trained Models

- Save your trained model to S3
- Can deploy two ways:
 - Persistent endpoint for making individual predictions on demand
 - SageMaker Batch Transform to get predictions for an entire dataset
- Lots of cool options
 - Inference Pipelines for more complex processing
 - SageMaker Neo for deploying to edge devices
 - Elastic Inference for accelerating deep learning models
 - Automatic scaling (increase # of endpoints as needed)
 - Shadow Testing evaluates new models against currently deployed model to catch errors



SageMaker Input Modes

- S3 File Mode
 - Default; **copies** training data from S3 to local directory in Docker container
- S3 Fast File Mode
 - Akin to “pipe mode”; training can begin without waiting to download data
 - Can do random access, but works best with sequential access
- Pipe Mode
 - Streams data directly from S3
 - Mainly replaced by Fast File
- Amazon S3 Express One Zone
 - High-performance storage class in one AZ
 - Works with file, fast file, and pipe modes
- Amazon FSx for Lustre
 - Scales to 100's of GB of throughput and millions of IOPS with low latency
 - Single AZ, requires VPC
- Amazon EFS
 - Requires data to be in EFS already, requires VPC



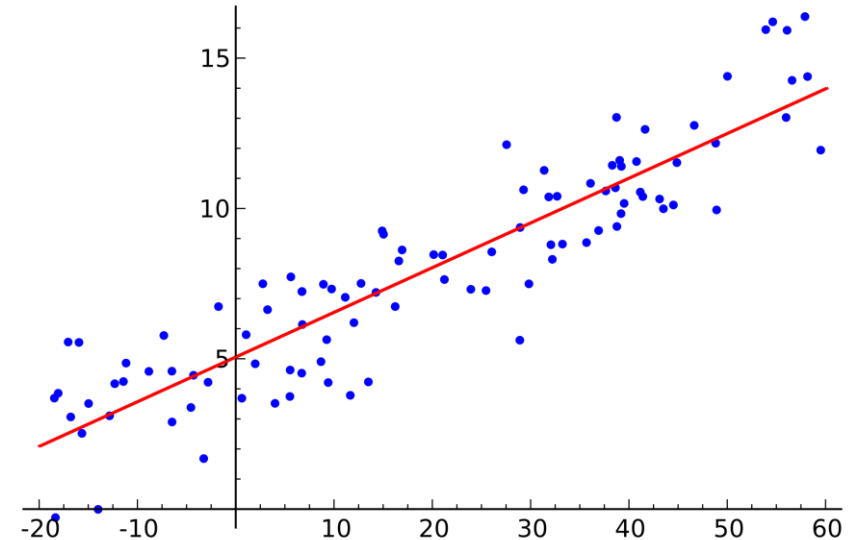
<https://docs.aws.amazon.com/sagemaker/latest/dg/model-access-training-data.html>

SageMaker's Built-In Algorithms

Linear Learner

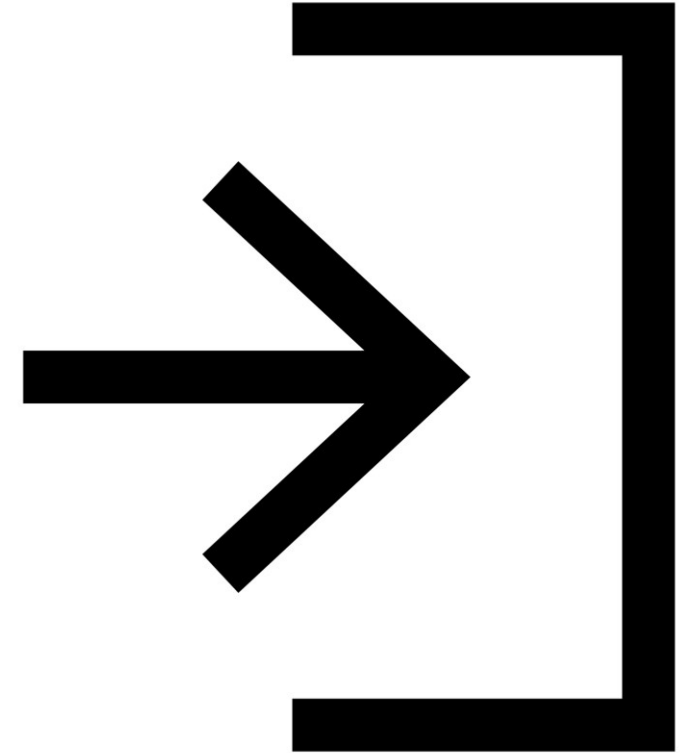
Linear Learner: What's it for?

- Linear regression
 - Fit a line to your training data
 - Predications based on that line
- Can handle both regression (numeric) predictions and classification predictions
 - For classification, a linear threshold function is used.
 - Can do binary or multi-class



Linear Learner: What training input does it expect?

- RecordIO-wrapped protobuf
 - Float32 data only!
- CSV
 - First column assumed to be the label
- File or Pipe mode both supported



Linear Learner: How is it used?

- Preprocessing
 - Training data must be *normalized* (so all features are weighted the same)
 - Linear Learner can do this for you automatically
 - Input data should be *shuffled*
- Training
 - Uses stochastic gradient descent
 - Choose an optimization algorithm (Adam, AdaGrad, SGD, etc)
 - Multiple models are optimized in parallel
 - Tune L1, L2 regularization
- Validation
 - Most optimal model is selected



Linear Learner: Important Hyperparameters

- `Balance_multiclass_weights`
 - Gives each class equal importance in loss functions
- `Learning_rate`, `mini_batch_size`
- `L1`
 - Regularization
- `Wd`
 - Weight decay (L2 regularization)
- `target_precision`
 - Use with `binary_classifier_model_selection_criteria` set to `recall_at_target_precision`
 - Holds precision at this value while maximizing recall
- `target_recall`
 - Use with `binary_classifier_model_selection_criteria` set to `precision_at_target_recall`
 - Holds recall at this value while maximizing precision



Linear Learner: Instance Types

- Training
 - Single or multi-machine CPU or GPU
 - Multi-GPU does not help



Example

jupyter linear_learner_mnist (autosaved)

File Edit View Insert Cell Kernel Widgets Help Trusted | conda_python3

Run nbdiff

Data inspection

Once the dataset is imported, it's typical as part of the machine learning process to inspect the data, understand the distributions, and determine what type(s) of preprocessing might be needed. You can perform those tasks right here in the notebook. As an example, let's go ahead and look at one of the digits that is part of the dataset.

In [3]:

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams["figure.figsize"] = (2,10)

def show_digit(img, caption='', subplot=None):
    if subplot==None:
        _,(subplot)=plt.subplots(1,1)
    imgr=img.reshape((28,28))
    subplot.axis('off')
    subplot.imshow(imgr, cmap='gray')
    plt.title(caption)

show_digit(train_set[0][30], 'This is a {}'.format(train_set[1][30]))
```

This is a 3



Data conversion

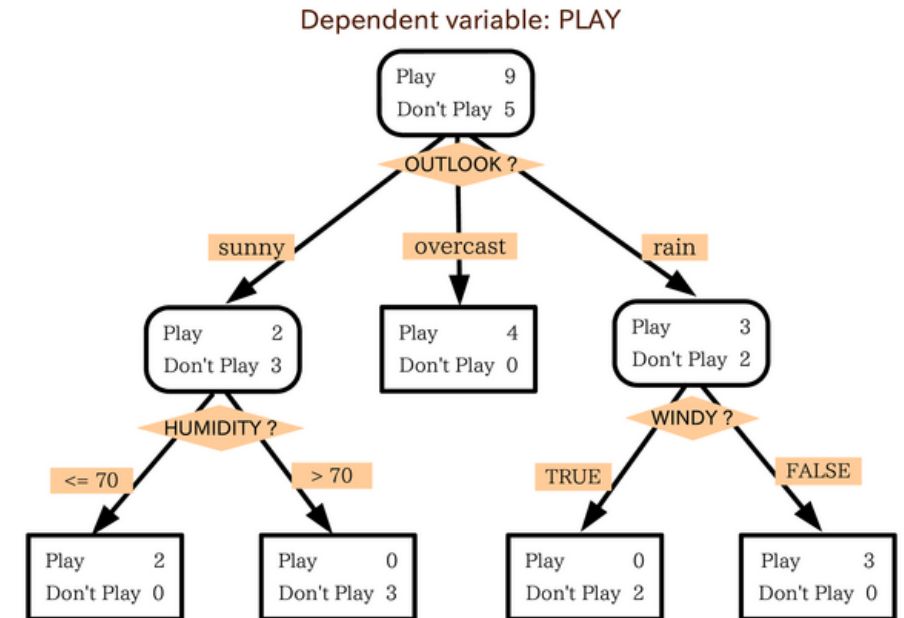
Since algorithms have particular input and output requirements, converting the dataset is also part of the process that a data scientist goes through prior to initiating training. In this particular case, the Amazon SageMaker implementation of Linear Learner takes recordIO-wrapped protobuf, where the data we have today is a pickle-sized numpy array on disk.

Most of the conversion effort is handled by the Amazon SageMaker Python SDK, imported as `sagemaker` below.

XGBoost

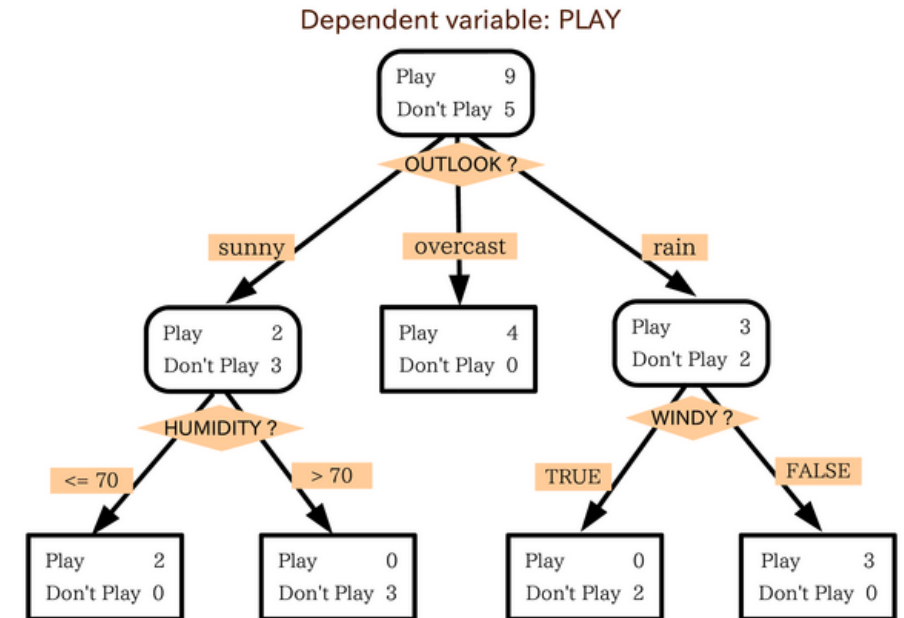
XGBoost: What's it for?

- eXtreme Gradient Boosting
 - Boosted group of decision trees
 - New trees made to correct the errors of previous trees
 - Uses gradient descent to minimize loss as new trees are added
- It's been winning a lot of Kaggle competitions
 - And it's fast, too
- Can be used for classification
- And also for regression
 - Using regression trees



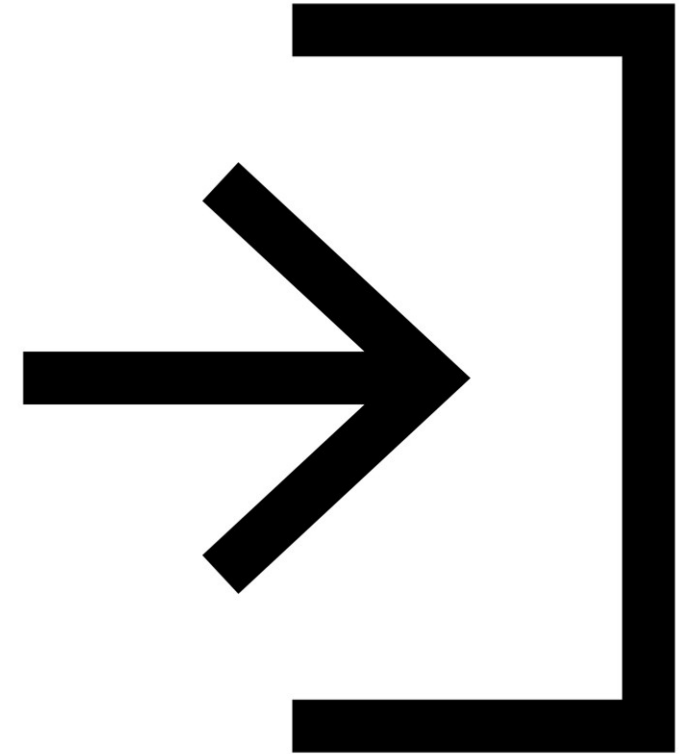
XGBoost: What's it for?

- eXtreme Gradient Boosting
 - Boosted group of decision trees
 - New trees made to correct the errors of previous trees
 - Uses gradient descent to minimize loss as new trees are added
- It's been winning a lot of Kaggle competitions
 - And it's fast, too
- Can be used for classification
- And also for regression
 - Using regression trees



XGBoost: What training input does it expect?

- XGBoost is weird, since it's not made for SageMaker. It's just open source XGBoost
- So, it takes CSV or libsvm input.
- AWS recently extended it to accept recordIO-protobuf and Parquet as well.



XGBoost: How is it used?

- Models are serialized/deserialized with Pickle
- Can use as a framework within notebooks
 - Sagemaker.xgboost
- Or as a built-in SageMaker algorithm



XGBoost: Important Hyperparameters

- There are a lot of them. A few:
- Subsample
 - Prevents overfitting
- Eta
 - Step size shrinkage, prevents overfitting
- Gamma
 - Minimum loss reduction to create a partition; larger = more conservative
- Alpha
 - L1 regularization term; larger = more conservative
- Lambda
 - L2 regularization term; larger = more conservative



XGBoost: Important Hyperparameters

- **eval_metric**
 - Optimize on AUC, error, rmse...
 - For example, if you care about false positives more than accuracy, you might use AUC here
- **scale_pos_weight**
 - Adjusts balance of positive and negative weights
 - Helpful for unbalanced classes
 - Might set to $\frac{\text{sum(negative cases)}}{\text{sum(positive cases)}}$
- **max_depth**
 - Max depth of the tree
 - Too high and you may overfit



XGBoost: Instance Types

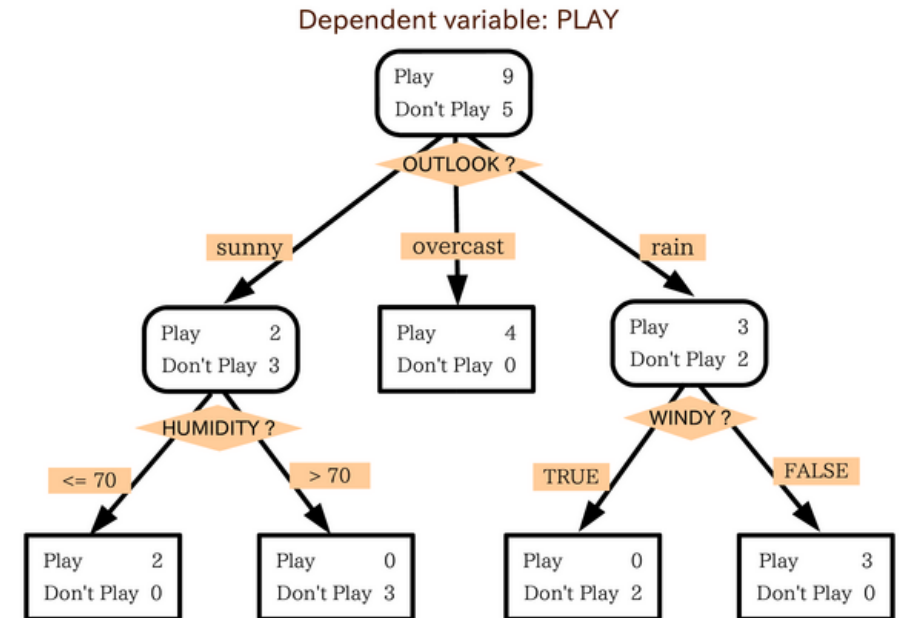
- Is memory-bound, not compute-bound
- So, **M5** is a good choice
- As of XGBoost 1.2, single-instance GPU training is available
 - For **example P2, P3**
 - Must set `tree_method` hyperparameter to `gpu_hist`
 - Trains more quickly and can be more cost effective.
- XGBoost 1.2-2
 - P2, P3, G4dn, G5
- XGBoost 1.5+: Distributed GPU training now available
 - Must set `use_dask_gpu_training` to `true`
 - Set `distribution` to `fully_replicated` in `TrainingInput`
 - Only works with `csv` or `parquet` input



LightGBM

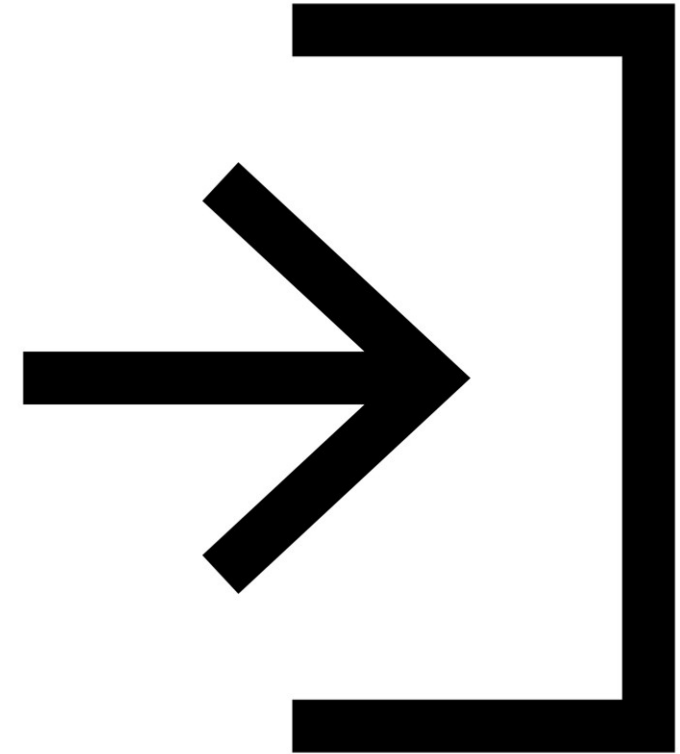
LightGBM: What's it for?

- Gradient Boosting Decision Tree
 - Kinda like XGBoost
 - Even more like CatBoost
 - Extended with Gradient-based One-Side Sampling and Exclusive Feature Bundling
- Predicts target variables with an ensemble of estimates from simpler models
- Classification, regression, or ranking



LightGBM: What training input does it expect?

- Requires txt/csv for training and for inference.
- Training and optional validation channels may be provided



LightGBM: Important Hyperparameters

- There are a lot of them. A few:
- Learning_rate
- Num_leaves
 - Max leaves per tree
- Feature_fraction
 - Subset of features per tree
- Bagging_fraction
 - Similar, but randomly sampled
- Bagging_freq
 - How often bagging is done
- Max_depth
- Min_data_in_leaf
 - Minimum amount of data in one leaf; can address overfitting



LightGBM: Instance Types

- Single or multi-instance CPU training
 - Define instances with `instance_count`
- Memory-bound algorithm
 - Choose general purpose over compute-optimized
 - M5, not C5
 - Be sure you have enough memory



Seq2Seq

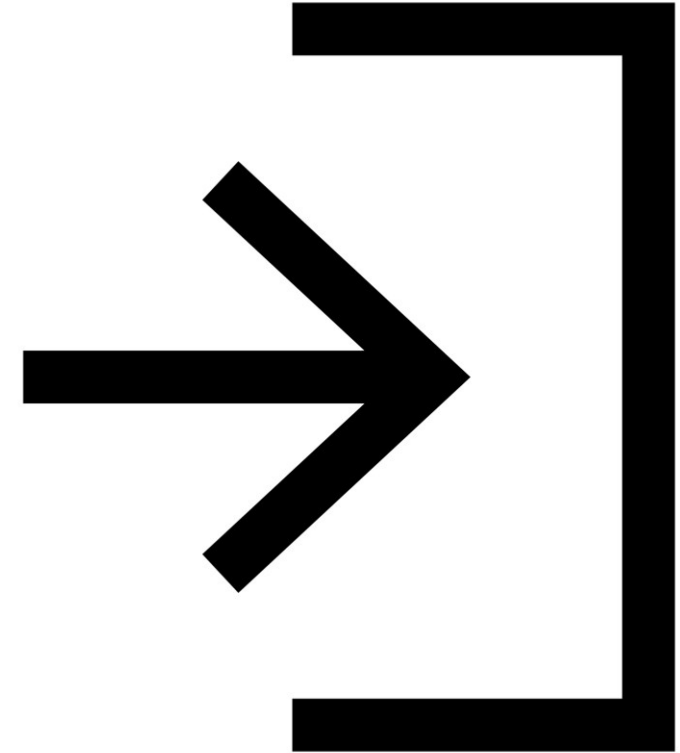
Seq2Seq: What's it for?

- Input is a sequence of tokens, output is a sequence of tokens
- Machine Translation
- Text summarization
- Speech to text
- Implemented with RNN's and CNN's with attention



Seq2Seq: What training input does it expect?

- RecordIO-Protobuf
 - Tokens must be integers (this is unusual, since most algorithms want floating point data.)
- Start with tokenized text files
- Convert to protobuf using sample code
 - Packs into integer tensors with vocabulary files
 - A lot like the TF/IDF lab we did earlier.
- Must provide training data, validation data, and vocabulary files.



Seq2Seq: How is it used?

- Training for machine translation can take days, even on SageMaker
- Pre-trained models are available
 - See the example notebook
- Public training datasets are available for specific translation tasks



Seq2Seq: Important Hyperparameters

- Batch_size
- Optimizer_type (adam, sgd, rmsprop)
- Learning_rate
- Num_layers_encoder
- Num_layers_decoder
- Can optimize on:
 - Accuracy
 - Vs. provided validation dataset
 - BLEU score
 - Compares against multiple reference translations
 - Perplexity
 - Cross-entropy



Seq2Seq: Instance Types

- Can only use GPU instance types (P3 for example)
- Can only use a single machine for training
 - But can use multi-GPU's on one machine



DeepAR

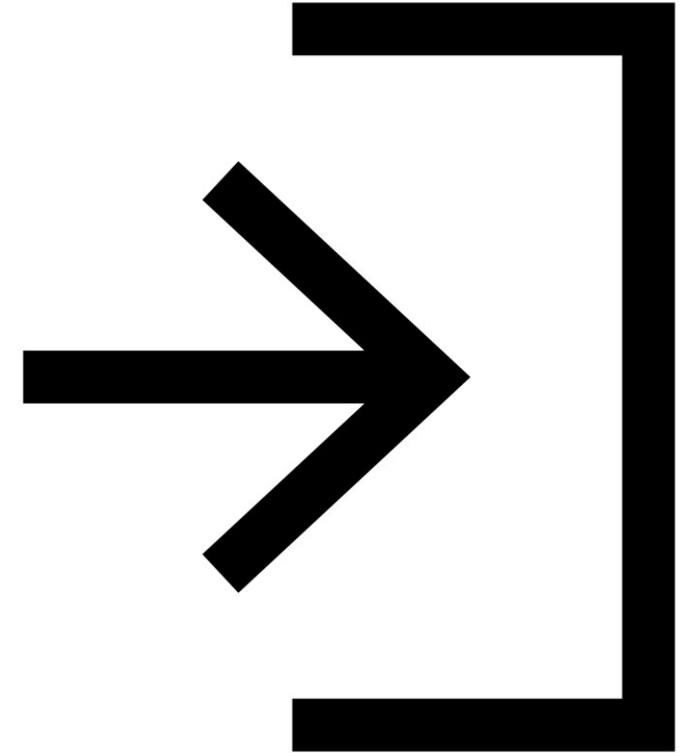
DeepAR: What's it for?

- Forecasting one-dimensional time series data
- Uses RNN's
- Allows you to train the same model over several related time series
- Finds frequencies and seasonality



DeepAR: What training input does it expect?

- JSON lines format
 - Gzip or Parquet
- Each record must contain:
 - Start: the starting time stamp
 - Target: the time series values
- Each record can contain:
 - Dynamic_feat: dynamic features (such as, was a promotion applied to a product in a time series of product purchases)
 - Cat: categorical features



```
{"start": "2009-11-01 00:00:00", "target": [4.3, "NaN", 5.1, ...], "cat": [0, 1], "dynamic_feat": [[1.1, 1.2, 0.5, ...]]}  
{"start": "2012-01-30 00:00:00", "target": [1.0, -5.0, ...], "cat": [2, 3], "dynamic_feat": [[1.1, 2.05, ...]]}  
{"start": "1999-01-30 00:00:00", "target": [2.0, 1.0], "cat": [1, 4], "dynamic_feat": [[1.3, 0.4]]}
```


DeepAR: How is it used?

- Always include entire time series for training, testing, and inference
- Use entire dataset as training set, remove last time points for testing. Evaluate on withheld values.
- Don't use very large values for prediction length (> 400)
- Train on many time series and not just one when possible



DeepAR: Important Hyperparameters

- Context_length
 - Number of time points the model sees before making a prediction
 - Can be smaller than seasonalities; the model will lag one year anyhow.
- Epochs
- mini_batch_size
- Learning_rate
- Num_cells



DeepAR: Instance Types

- Can use CPU or GPU
- Single or multi machine
- Start with CPU (ml.c4.2xlarge, ml.c4.4xlarge)
- Move up to GPU if necessary
 - Only helps with larger models
 - Or with large mini-batch sizes (>512)
- CPU-only for inference
- May need larger instances for tuning



BlazingText

BlazingText: What's it for?

- Text classification
 - Predict labels for a sentence
 - Useful in web searches, information retrieval
 - Supervised
- Word2vec
 - Creates a vector representation of words
 - Semantically similar words are represented by vectors close to each other
 - This is called a *word embedding*
 - It is useful for NLP, but is not an NLP algorithm in itself!
 - Used in machine translation, sentiment analysis
 - Remember it only works on individual words, not sentences or documents



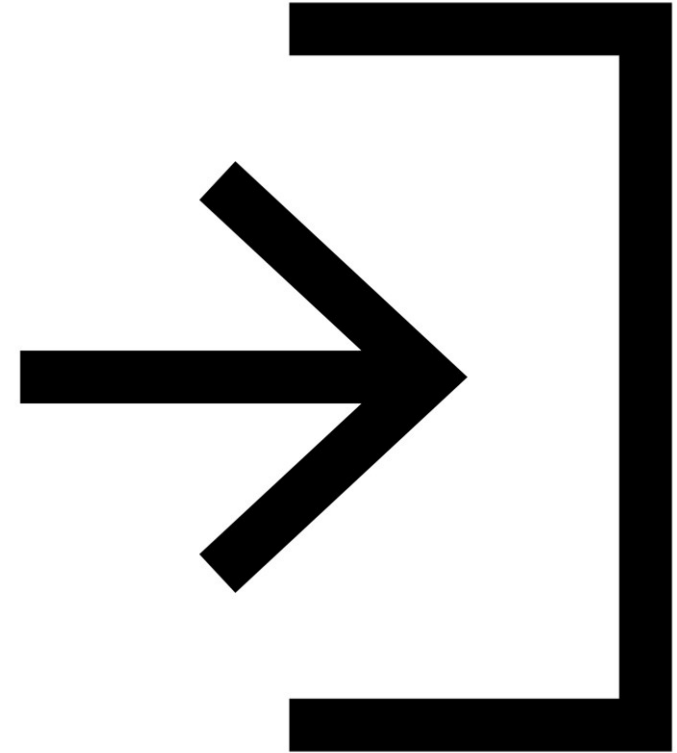
BlazingText: What training input does it expect?

- For supervised mode (text classification):
 - One sentence per line
 - First “word” in the sentence is the string `__label__` followed by the label
- Also, “augmented manifest text format”
- Word2vec just wants a text file with one training sentence per line.

`__label__4` linux ready for prime time , intel says , despite all the linux hype , the open-source movement has yet to make a huge splash in the desktop market . that may be about to change , thanks to chipmaking giant intel corp .

`__label__2` bowled by the slower one again , kolkata , november 14 the past caught up with sourav ganguly as the indian skippers return to international cricket was short lived .

```
{"source":"linux ready for prime time , intel says , despite all the linux hype", "label":1}  
{"source":"bowled by the slower one again , kolkata , november 14 the past caught up with  
sourav ganguly", "label":2}
```



BlazingText: How is it used?

- Word2vec has multiple modes
 - Cbow (Continuous Bag of Words)
 - Skip-gram
 - Batch skip-gram
 - Distributed computation over many CPU nodes



BlazingText: Important Hyperparameters

- Word2vec:
 - Mode (batch_skipgram, skipgram, cbow)
 - Learning_rate
 - Window_size
 - Vector_dim
 - Negative_samples
- Text classification:
 - Epochs
 - Learning_rate
 - Word_ngrams
 - Vector_dim



BlazingText: Instance Types

- For cbow and skipgram, recommend a single ml.p3.2xlarge
 - Any single CPU or single GPU instance will work
- For batch_skipgram, can use single or multiple CPU instances
- For text classification, C5 recommended if less than 2GB training data. For larger data sets, use a single GPU instance (ml.p2.xlarge or ml.p3.2xlarge)



Object2Vec

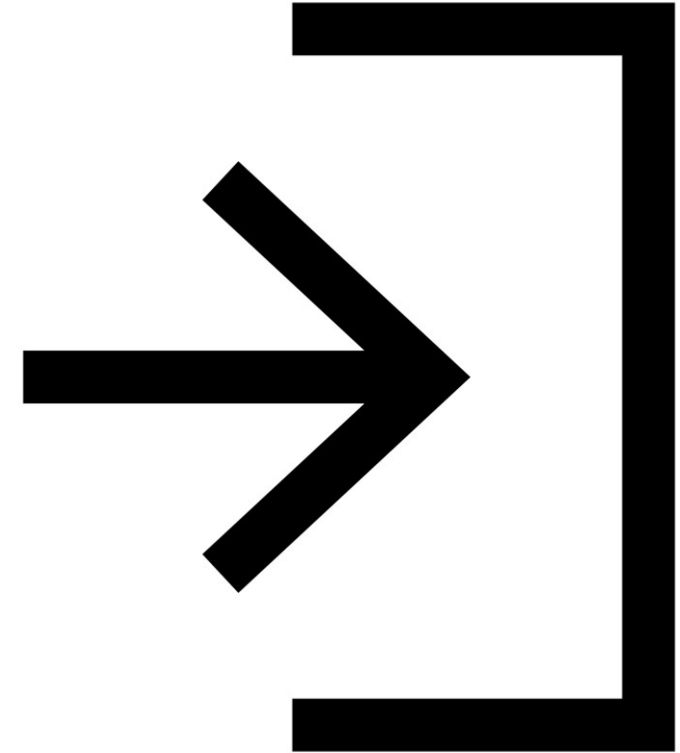
Object2Vec: What's it for?

- Remember word2vec from Blazing Text? It's like that, but arbitrary objects
- It creates low-dimensional dense embeddings of high-dimensional objects
- It is basically word2vec, generalized to handle things other than words.
- Compute nearest neighbors of objects
- Visualize clusters
- Genre prediction
- Recommendations (similar items or users)



Object2Vec: What training input does it expect?

- Data must be tokenized into integers
- Training data consists of pairs of tokens and/or sequences of tokens
 - Sentence – sentence
 - Labels-sequence (genre to description?)
 - Customer-customer
 - Product-product
 - User-item



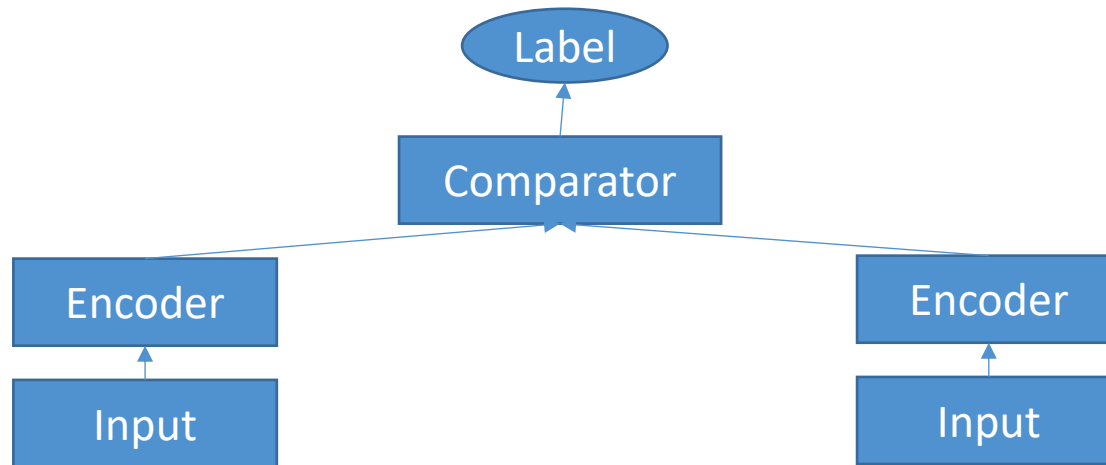
```
{"label": 0, "in0": [6, 17, 606, 19, 53, 67, 52, 12, 5, 10, 15, 10178, 7, 33, 652, 80, 15, 69, 821, 4], "in1": [16, 21, 13, 45, 14, 9, 80, 59, 164, 4]}
```

```
{"label": 1, "in0": [22, 1016, 32, 13, 25, 11, 5, 64, 573, 45, 5, 80, 15, 67, 21, 7, 9, 107, 4], "in1": [22, 32, 13, 25, 1016, 573, 3252, 4]}
```

```
{"label": 1, "in0": [774, 14, 21, 206], "in1": [21, 366, 125]}
```

Object2Vec: How is it used?

- Process data into JSON Lines and shuffle it
- Train with two input channels, two encoders, and a comparator
- Encoder choices:
 - Average-pooled embeddings
 - CNN's
 - Bidirectional LSTM
- Comparator is followed by a feed-forward neural network



Object2Vec: Important Hyperparameters

- The usual deep learning ones...
 - Dropout, early stopping, epochs, learning rate, batch size, layers, activation function, optimizer, weight decay
- Enc1_network, enc2_network
 - Choose hcnn, bilstm, pooled_embedding



Object2Vec: Instance Types

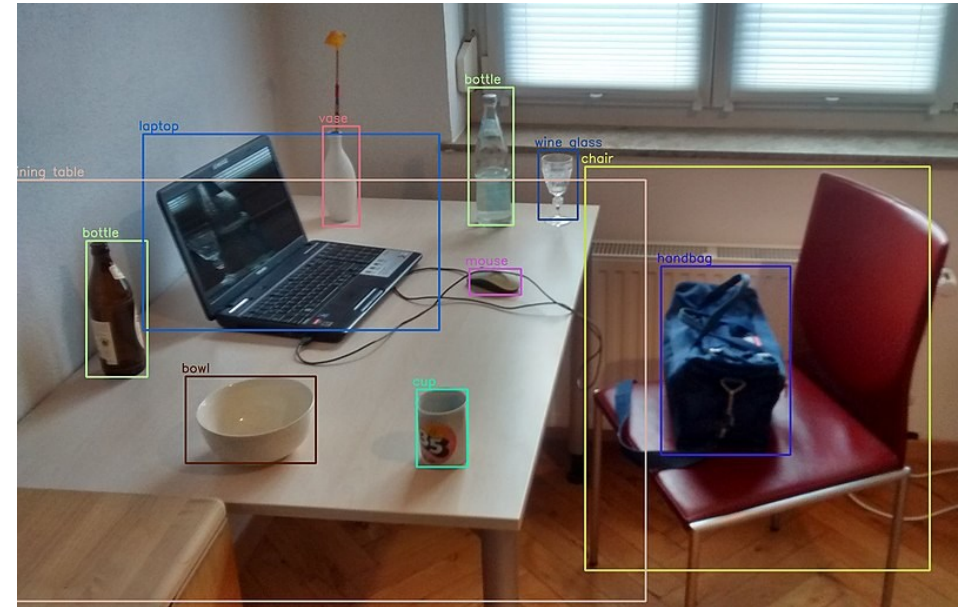
- Can only train on a single machine (CPU or GPU, multi-GPU OK)
 - ml.m5.2xlarge
 - ml.p2.xlarge
 - If needed, go up to ml.m5.4xlarge or ml.m5.12xlarge
 - GPU options: P2, P3, G4dn, G5
- Inference: use ml.p3.2xlarge
 - Use `INFERENC``_PREFERRED_MODE` environment variable to optimize for encoder embeddings rather than classification or regression.



Object Detection

Object Detection: What's it for?

- Identify all objects in an image with bounding boxes
- Detects and classifies objects with a single deep neural network
- Classes are accompanied by confidence scores
- Can train from scratch, or use pre-trained models based on ImageNet



(MTheiler) [CC BY-SA 4.0 (<https://creativecommons.org/licenses/by-sa/4.0/>)]

Object Detection: How is it used?

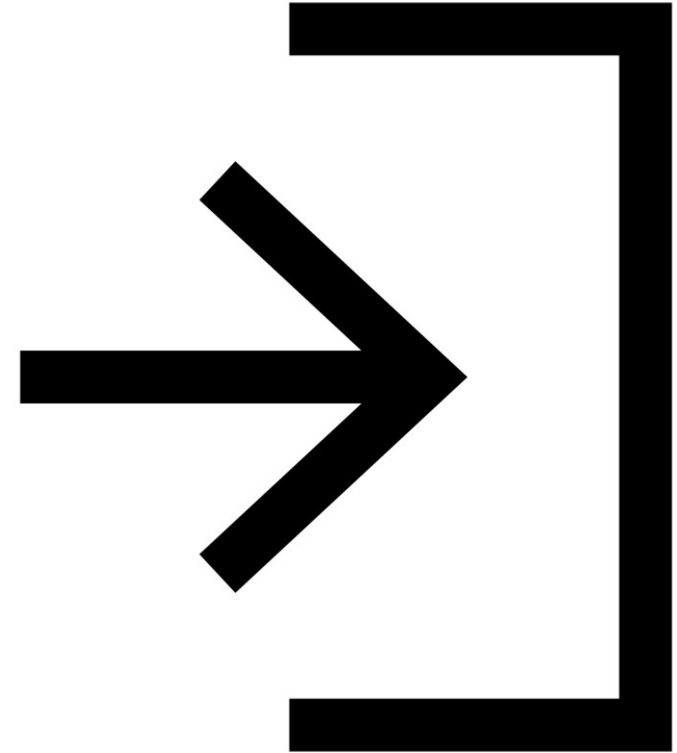
- Two variants: MXNet and Tensorflow
- Takes an image as input, outputs all instances of objects in the image with categories and confidence scores
- MXNet
 - Uses a CNN with the Single Shot multibox Detector (SSD) algorithm
 - The base CNN can be VGG-16 or ResNet-50
 - Transfer learning mode / incremental training
 - Use a pre-trained model for the base network weights instead of random initial weights
 - Uses flip, rescale, and jitter internally to avoid overfitting
- Tensorflow
 - Uses ResNet, EfficientNet, MobileNet models from the TensorFlow Model Garden



Object Detection: What training input does it expect?

- MXNet: RecordIO or image format (jpg or png)
- With image format, supply a JSON file for annotation data for each image

```
"file": "your_image_directory/sample_image1.jpg",  
"image_size": [  
  {  
    "width": 500,  
    "height": 400,  
    "depth": 3  
  }  
],  
"annotations": [  
  {  
    "class_id": 0,  
    "left": 111,  
    "top": 134,  
    "width": 61,  
    "height": 128  
  },  
],  
"categories": [  
  {  
    "class_id": 0,  
    "name": "dog"  
  },  
]  
}
```



Object Detection: Important Hyperparameters

- Mini_batch_size
- Learning_rate
- Optimizer
 - Sgd, adam, rmsprop, adadelata



Object Detection: Instance Types

- Use GPU instances for training (multi-GPU and multi-machine OK)
 - ml.p2.xlarge, ml.p2.16xlarge, ml.p3.2xlarge, ml.p3.16xlarge, G4dn, G5
- Use CPU or GPU for inference
 - M5, P2, P3, G4dn all OK



Image Classification

Image Classification: What's it for?

- Assign one or more labels to an image
- Doesn't tell you where objects are, just what objects are in the image



Image Classification: How is it used?

- Separate algorithms for MXNet and Tensorflow
- MXNet:
 - Full training mode
 - Network initialized with random weights
 - Transfer learning mode
 - Initialized with pre-trained weights
 - The top fully-connected layer is initialized with random weights
 - Network is fine-tuned with new training data
 - Default image size is 3-channel 224x224 (ImageNet's dataset)
- Tensorflow: Uses various Tensorflow Hub models (MobileNet, Inception, ResNet, EfficientNet)
 - Top classification layer is available for fine tuning or further training



Image Classification: Important Hyperparameters

- The usual suspects for deep learning
 - Batch size, learning rate, optimizer
- Optimizer-specific parameters
 - Weight decay, beta 1, beta 2, eps, gamma
 - Slightly different between MXNet and Tensorflow versions



Image Classification: Instance Types

- GPU instances for training (ml.p2, p3, g4dn, g5) Multi-GPU and multi-machine OK.
- CPU or GPU for inference (m5, p2, p3, g4dn, g5)



Semantic Segmentation

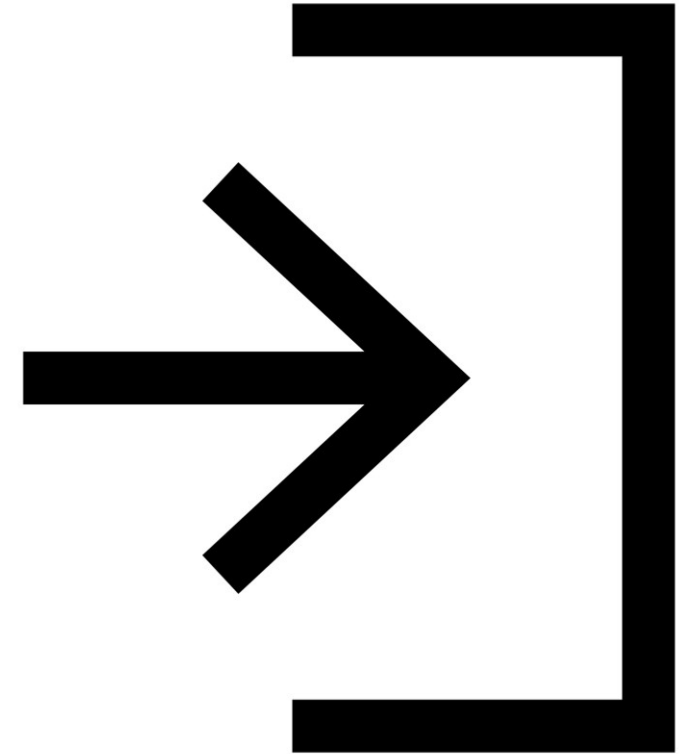
Semantic Segmentation: What's it for?

- Pixel-level object classification
- Different from image classification – that assigns labels to whole images
- Different from object detection – that assigns labels to bounding boxes
- Useful for self-driving vehicles, medical imaging diagnostics, robot sensing
- Produces a *segmentation mask*



Semantic Segmentation: What training input does it expect?

- JPG Images and PNG annotations
- For both training and validation
- Label maps to describe annotations
- Augmented manifest image format supported for Pipe mode.
- JPG images accepted for inference



Semantic Segmentation: How is it used?

- Built on MXNet Gluon and Gluon CV
- Choice of 3 algorithms:
 - Fully-Convolutional Network (FCN)
 - Pyramid Scene Parsing (PSP)
 - DeepLabV3
- Choice of backbones:
 - ResNet50
 - ResNet101
 - Both trained on ImageNet
- Incremental training, or training from scratch, supported too



Semantic Segmentation: Important Hyperparameters

- Epochs, learning rate, batch size, optimizer, etc
- Algorithm
- Backbone



Image Classification: Instance Types

- GPU instances for training (ml.p2, p3, g4dn, g5) Multi-GPU and multi-machine OK.
- CPU or GPU for inference (m5, p2, p3, g4dn, g5)



Random Cut Forest

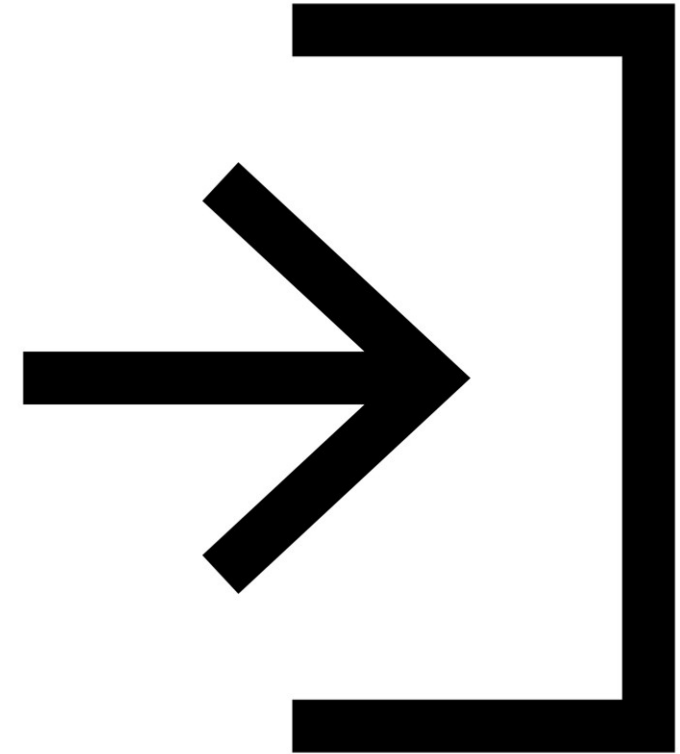
Random Cut Forest: What's it for?

- Anomaly detection
- Unsupervised
- Detect unexpected spikes in time series data
- Breaks in periodicity
- Unclassifiable data points
- Assigns an anomaly score to each data point
- Based on an algorithm developed by Amazon that they seem to be very proud of!



Random Cut Forest: What training input does it expect?

- RecordIO-protobuf or CSV
- Can use File or Pipe mode on either
- Optional test channel for computing accuracy, precision, recall, and F1 on labeled data (anomaly or not)



Random Cut Forest: How is it used?

- Creates a forest of trees where each tree is a partition of the training data; looks at expected change in complexity of the tree as a result of adding a point into it
- Data is sampled randomly
- Then trained
- RCF shows up in Kinesis Analytics as well; it can work on streaming data too.



Random Cut Forest: Important Hyperparameters

- Num_trees
 - Increasing reduces noise
- Num_samples_per_tree
 - Should be chosen such that $1/\text{num_samples_per_tree}$ approximates the ratio of anomalous to normal data



Random Cut Forest: Instance Types

- Does not take advantage of GPUs
- Use M4, C4, or C5 for training
- ml.c5.xl for inference



Neural Topic Model

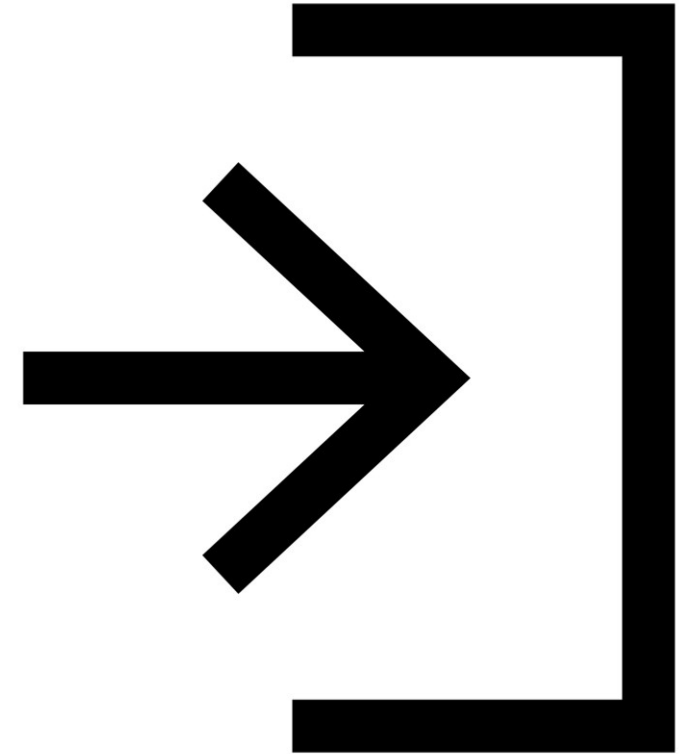
Neural Topic Model: What's it for?

- Organize documents into topics
- Classify or summarize documents based on topics
- It's not just TF/IDF
 - “bike”, “car”, “train”, “mileage”, and “speed” might classify a document as “transportation” for example (although it wouldn't know to call it that)
- Unsupervised
 - Algorithm is “Neural Variational Inference”



Neural Topic Model: What training input does it expect?

- Four data channels
 - “train” is required
 - “validation”, “test”, and “auxiliary” optional
- recordIO-protobuf or CSV
- Words must be tokenized into integers
 - Every document must contain a count for every word in the vocabulary in CSV
 - The “auxiliary” channel is for the vocabulary
- File or pipe mode



Neural Topic Model: How is it used?

- You define how many topics you want
- These topics are a latent representation based on top ranking words
- One of two topic modeling algorithms in SageMaker – you can try them both!



Neural Topic Model: Important Hyperparameters

- Lowering mini_batch_size and learning_rate can reduce validation loss
 - At expense of training time
- Num_topics



Neural Topic Model: Instance Types

- GPU or CPU
 - GPU recommended for training
 - CPU OK for inference
 - CPU is cheaper



LDA

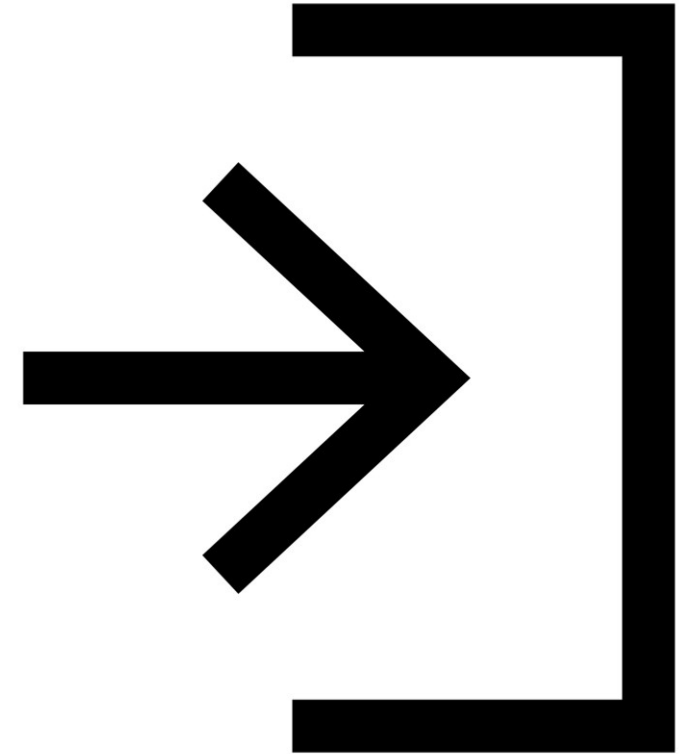
LDA: What's it for?

- Latent Dirichlet Allocation
- Another topic modeling algorithm
 - Not deep learning
- Unsupervised
 - The topics themselves are unlabeled; they are just groupings of documents with a shared subset of words
- Can be used for things other than words
 - Cluster customers based on purchases
 - Harmonic analysis in music



LDA: What training input does it expect?

- Train channel, optional test channel
- recordIO-protobuf or CSV
- Each document has counts for every word in vocabulary (in CSV format)
- Pipe mode only supported with recordIO



LDA: How is it used?

- Unsupervised; generates however many topics you specify
- Optional test channel can be used for scoring results
 - Per-word log likelihood
- Functionally similar to NTM, but CPU-based
 - Therefore maybe cheaper / more efficient



LDA: Important Hyperparameters

- Num_topics
- Alpha0
 - Initial guess for concentration parameter
 - Smaller values generate sparse topic mixtures
 - Larger values (>1.0) produce uniform mixtures



LDA: Instance Types

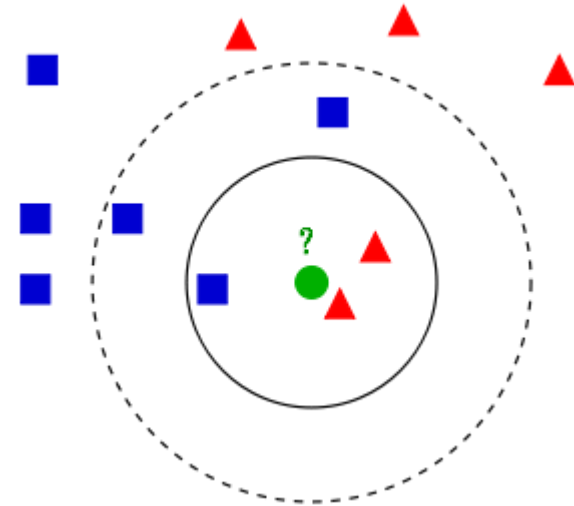
- Single-instance CPU training



KNN

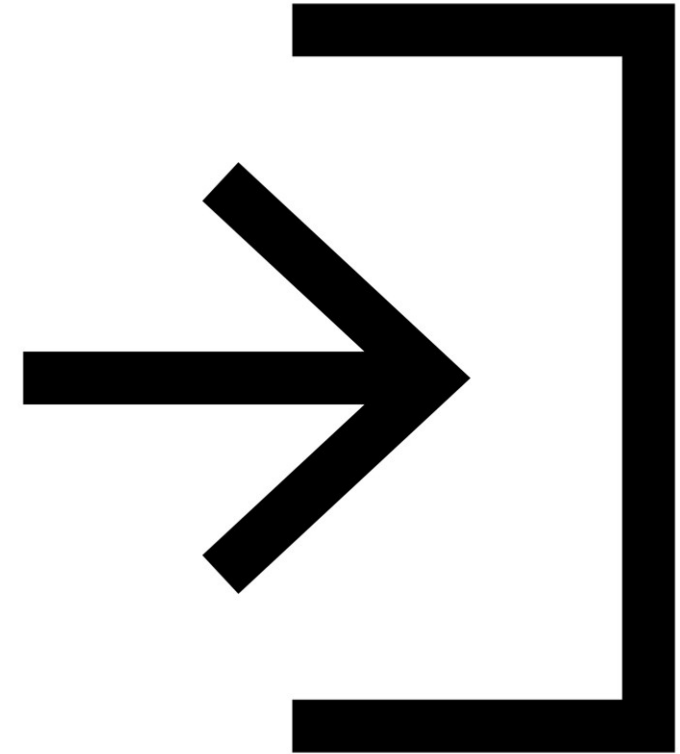
KNN: What's it for?

- K-Nearest-Neighbors
- Simple classification or regression algorithm
- Classification
 - Find the K closest points to a sample point and return the most frequent label
- Regression
 - Find the K closest points to a sample point and return the average value



KNN: What training input does it expect?

- Train channel contains your data
- Test channel emits accuracy or MSE
- recordIO-protobuf or CSV training
 - First column is label
- File or pipe mode on either



KNN: How is it used?

- Data is first sampled
- SageMaker includes a dimensionality reduction stage
 - Avoid sparse data (“curse of dimensionality”)
 - At cost of noise / accuracy
 - “sign” or “fjlt” methods
- Build an index for looking up neighbors
- Serialize the model
- Query the model for a given K



KNN: Important Hyperparameters

- K!
- Sample_size



KNN: Instance Types

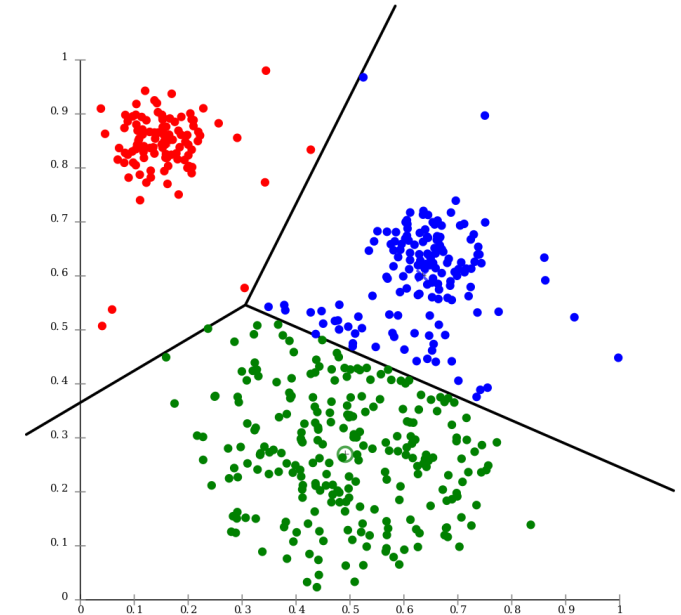
- Training on CPU or GPU
 - Ml.m5.2xlarge
 - Ml.p2.xlarge
- Inference
 - CPU for lower latency
 - GPU for higher throughput on large batches



K-Means

K-Means: What's it for?

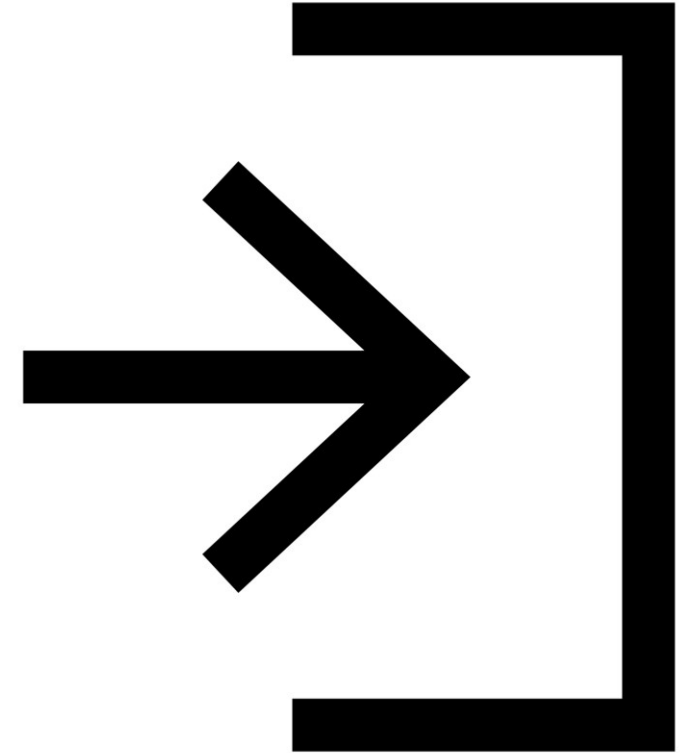
- Unsupervised clustering
- Divide data into K groups, where members of a group are as similar as possible to each other
 - You define what “similar” means
 - Measured by Euclidean distance
- Web-scale K-Means clustering



Chire [CC BY-SA 3.0 (<https://creativecommons.org/licenses/by-sa/3.0/>)]

K-Means: What training input does it expect?

- Train channel, optional test
 - Train ShardedByS3Key, test FullyReplicate
- recordIO-protobuf or CSV
- File or Pipe on either



K-Means: How is it used?

- Every observation mapped to n-dimensional space (n = number of features)
- Works to optimize the center of K clusters
 - “extra cluster centers” may be specified to improve accuracy (which end up getting reduced to k)
 - $K = k \times x$
- Algorithm:
 - Determine initial cluster centers
 - Random or k-means++ approach
 - K-means++ tries to make initial clusters far apart
 - Iterate over training data and calculate cluster centers
 - Reduce clusters from K to k
 - Using Lloyd’s method with kmeans++



K-Means: Important Hyperparameters

- K!
 - Choosing K is tricky
 - Plot within-cluster sum of squares as function of K
 - Use “elbow method”
 - Basically optimize for tightness of clusters
- Mini_batch_size
- Extra_center_factor
- Init_method



K-Means: Instance Types

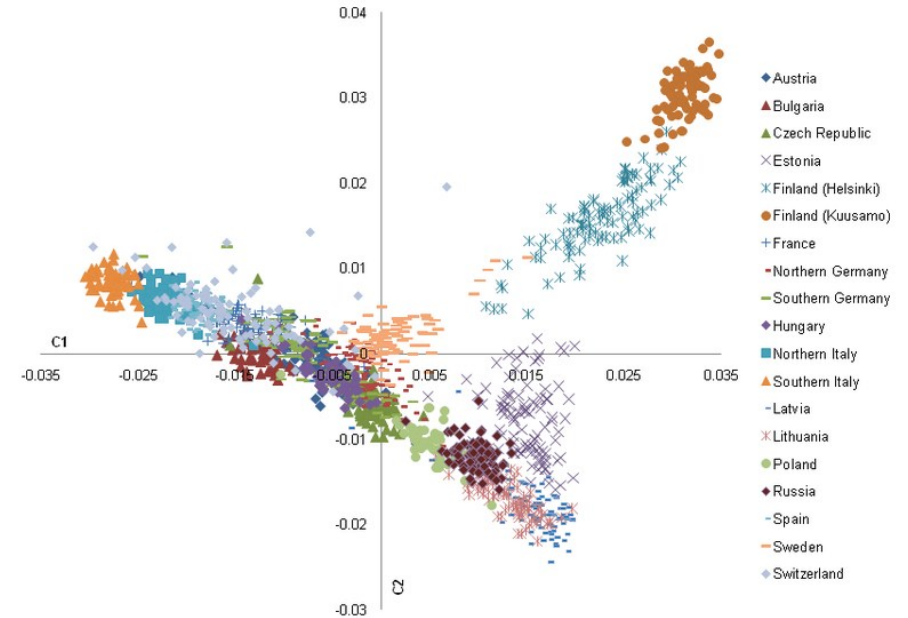
- CPU or GPU, but CPU recommended
 - Only one GPU per instance used on GPU
 - So use ml.g4dn.xlarge if you're going to use GPU
 - p2, p3, g4dn, and g4 supported



PCA

PCA: What's it for?

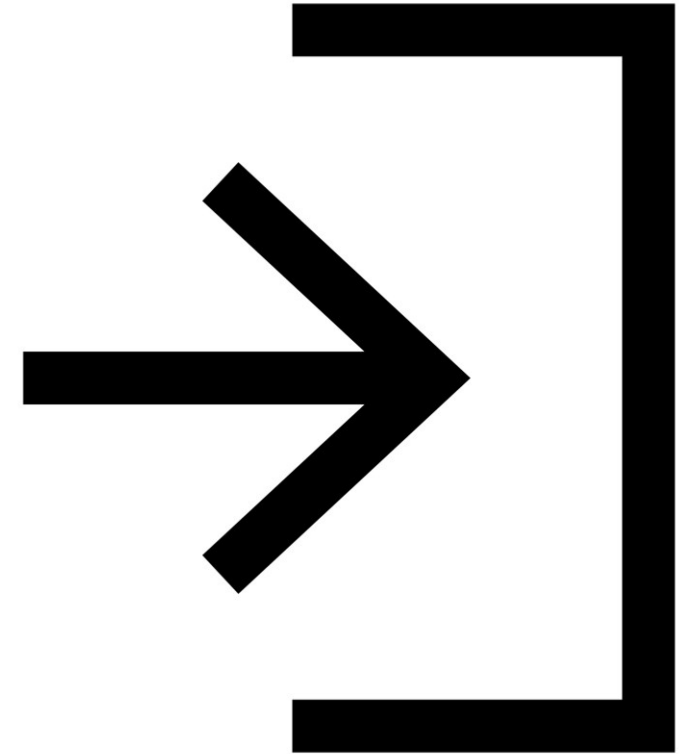
- Principal Component Analysis
- Dimensionality reduction
 - Project higher-dimensional data (lots of features) into lower-dimensional (like a 2D plot) while minimizing loss of information
 - The reduced dimensions are called components
 - First component has largest possible variability
 - Second component has the next largest...
- Unsupervised



Nelis M, Esko T, Ma'gi R, Zimprich F, Zimprich A, et al. (2009) [CC BY 2.5 (<https://creativecommons.org/licenses/by/2.5>)]

PCA: What training input does it expect?

- recordIO-protobuf or CSV
- File or Pipe on either



PCA: How is it used?

- Covariance matrix is created, then singular value decomposition (SVD)
- Two modes
 - Regular
 - For sparse data and moderate number of observations and features
 - Randomized
 - For large number of observations and features
 - Uses approximation algorithm



PCA: Important Hyperparameters

- Algorithm_mode
- Subtract_mean
 - Unbias data



PCA: Instance Types

- GPU or CPU
 - It depends “on the specifics of the input data”



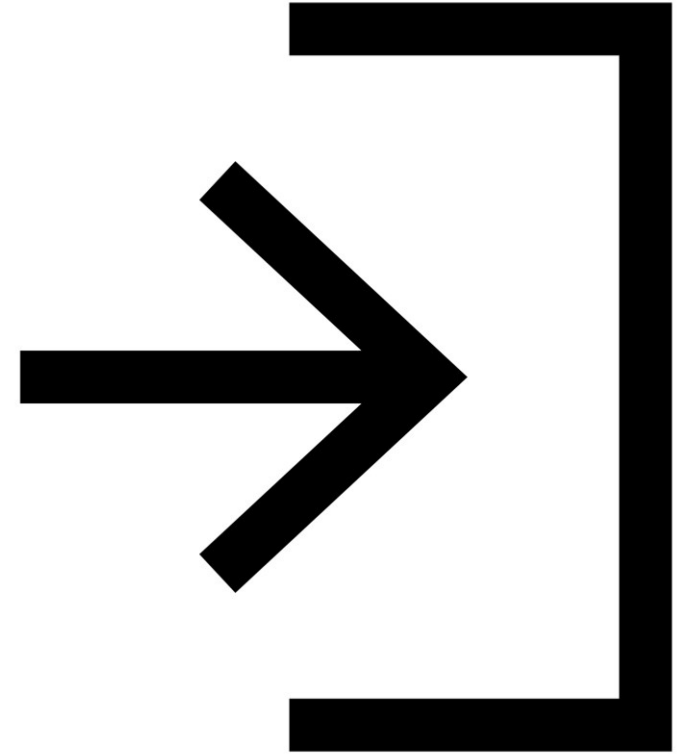
Factorization Machines

Factorization Machines: What's it for?

- Dealing with sparse data
 - Click prediction
 - Item recommendations
 - Since an individual user doesn't interact with most pages / products the data is sparse
- Supervised
 - Classification or regression
- Limited to pair-wise interactions
 - User -> item for example

Factorization Machines: What training input does it expect?

- recordIO-protobuf with Float32
 - Sparse data means CSV isn't practical



Factorization Machines: How is it used?

- Finds factors we can use to predict a classification (click or not? Purchase or not?) or value (predicted rating?) given a matrix representing some pair of things (users & items?)
- Usually used in the context of recommender systems



Factorization Machines: Important Hyperparameters

- Initialization methods for bias, factors, and linear terms
 - Uniform, normal, or constant
 - Can tune properties of each method



Factorization Machines: Instance Types

- CPU or GPU
 - CPU recommended
 - GPU only works with dense data



IP Insights

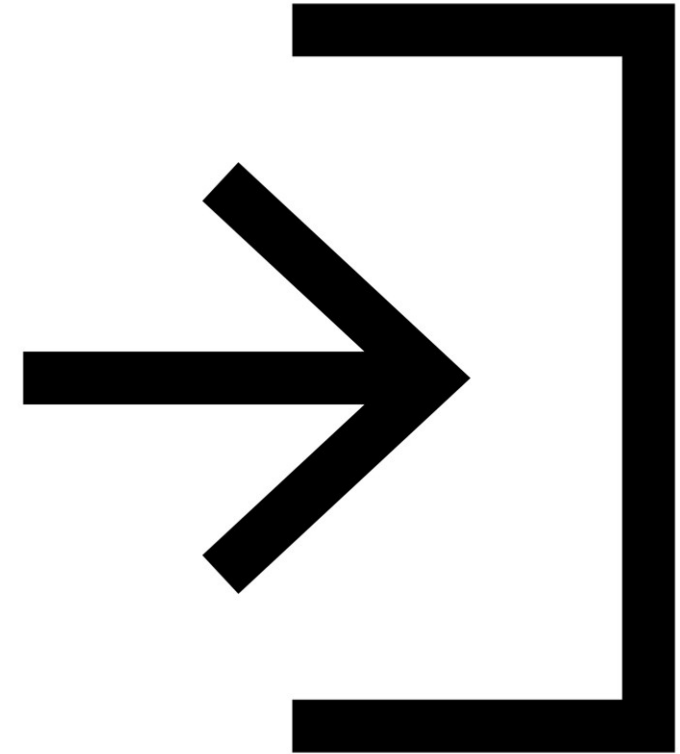
IP Insights: What's it for?

- Unsupervised learning of IP address usage patterns
- Identifies suspicious behavior from IP addresses
 - Identify logins from anomalous IP's
 - Identify accounts creating resources from anomalous IP's



IP Insights: What training input does it expect?

- User names, account ID's can be fed in directly; no need to pre-process
- Training channel, optional validation (computes AUC score)
- CSV only
 - Entity, IP



IP Insights: How is it used?

- Uses a neural network to learn latent vector representations of entities and IP addresses.
- Entities are hashed and embedded
 - Need sufficiently large hash size
- Automatically generates negative samples during training by randomly pairing entities and IP's



IP Insights: Important Hyperparameters

- Num_entity_vectors
 - Hash size
 - Set to twice the number of unique entity identifiers
- Vector_dim
 - Size of embedding vectors
 - Scales model size
 - Too large results in overfitting
- Epochs, learning rate, batch size, etc.



IP Insights: Instance Types

- CPU or GPU
 - GPU recommended
 - Ml.p3.2xlarge or higher
 - Can use multiple GPU's
 - Size of CPU instance depends on `vector_dim` and `num_entity_vectors`

